

Java Database Access Technologies

JDBC · JdbcTemplate · Spring Data JDBC · Hibernate · Spring Data JPA

Course Agenda

PART 1 — TECHNOLOGIES

- 1. Vanilla JDBC – the baseline
- 2. JdbcTemplate – removing boilerplate
- 3. Spring Data JDBC – lightweight ORM
- 4. Hibernate / Jakarta Persistence
- 5. Spring Data JPA – full ORM stack

- 6. Side-by-side comparison tables

PART 2 — PERFORMANCE

- 7. Benchmark methodology
- 8. Read / Write / Update / Delete metrics
- 9. Small vs Large dataset behaviour
- 10. Decision guide – when to use what

PART 1

Technology Deep Dives

1 · Vanilla JDBC — Introduction

Java Database Connectivity (JDBC) is the lowest-level Java API for relational databases, introduced in Java 1.1 (1997). It provides direct SQL execution with no abstraction layer.

✓ PROS

- Full control over SQL and execution
- Zero abstraction overhead
- Works with any JDBC-compliant driver
- Lightweight — no extra dependencies
- Best for tuned, complex queries

✗ CONS / Pain points

- Massive boilerplate (try/catch, close resources)
- Manual ResultSet mapping to objects
- SQL injection risk if not careful
- No transaction management helpers
- Hard to maintain for large projects

1 · Vanilla JDBC — Maven & Code Example

pom.xml (Maven)

```
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <version>42.7.3</version>
</dependency>
```

application.properties

```
# Not Spring-managed – manual in plain Java
jdbc.url=jdbc:postgresql://localhost:5432/mydb
jdbc.username=postgres
jdbc.password=secret
```

```
// Vanilla JDBC – typical read operation
String sql = "SELECT id, name FROM product WHERE id = ?";
try (Connection conn = DriverManager.getConnection(url, user, pass);
    PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setLong(1, productId);
    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) {
            return new Product(rs.getLong("id"),
                               rs.getString("name"));
        }
    }
} catch (SQLException e) {
    throw new RuntimeException(e); // handle properly!
}
// ⚠ Every query repeats: open conn → prepare → execute
// → map → close → catch – huge repetition!
```

2 · JdbcTemplate — Introduction

Spring's JdbcTemplate wraps JDBC, eliminating boilerplate while keeping full SQL control. It handles connection management, exception translation, and resource cleanup automatically.

JdbcTemplate vs. Vanilla JDBC — What changes?

Concern	Vanilla JDBC	JdbcTemplate
Connection open/close	Manual try-with-resources	Automatic ✓
Exception handling	Checked SQLException	DataAccessException (unchecked) ✓
ResultSet mapping	Manual rs.getString(...)	RowMapper / BeanPropertyRowMapper ✓
Transactions	Manual commit/rollback	@Transactional support ✓
SQL injection safety	Manual PreparedStatement	Built-in via ? params ✓
Lines of code (typical query)	~20–30 lines	~3–5 lines ✓

2 · JdbcTemplate — Maven, Config & Code

pom.xml (Maven)

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
</dependency>
```

application.properties

```
spring.datasource.url=jdbc:postgresql://localhost:5432/mydb
spring.datasource.username=postgres
spring.datasource.password=secret
spring.datasource.driver-class-name=org.postgresql.Driver
# Connection pool (HikariCP default)
spring.datasource.hikari.maximum-pool-size=10
```

```
@Repository
public class ProductRepository {
    private final JdbcTemplate jdbc;

    public Product findById(Long id) {
        return jdbc.queryForObject(
            "SELECT id, name FROM product WHERE id = ?",
            (rs, row) -> new Product(rs.getLong("id"),
                                     rs.getString("name")),
            id);
    }

    public int save(Product p) {
        return jdbc.update(
            "INSERT INTO product(name) VALUES (?)", p.getName());
    }
    // No try/catch, no conn.close() - Spring handles it all!
}
```

3 · Spring Data JDBC — Introduction

Spring Data JDBC is a lightweight ORM that maps Java objects to database rows without lazy loading, caching, or proxy magic. It follows DDD (Domain-Driven Design) principles.

JdbcTemplate vs. Spring Data JDBC		
Feature	JdbcTemplate	Spring Data JDBC
SQL writing	Always manual	Generated by framework; custom via @Query
Object mapping	RowMapper (manual)	Automatic via @Table / @Column
Repository CRUD	Write every method	CrudRepository gives save/find/delete free
Relationships	Manual JOINS	Embedded / AggregateReference (DDD)
Lazy loading	N/A	None — always eager (simple & predictable)
Session / cache	None	None — clean per-query
Complexity	Low	Low-medium

3 · Spring Data JDBC — Maven, Config & Code

pom.xml (Maven)

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jdbc</artifactId>
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
</dependency>
```

application.properties

```
spring.datasource.url=jdbc:postgresql://localhost:5432/mydb
spring.datasource.username=postgres
spring.datasource.password=secret
# DDL: validate | update | create-drop
spring.sql.init.mode=always
# Optional: schema.sql auto-run
spring.sql.init.schema-locations=classpath:schema.sql
```

```
// Entity
@Table("product")
public record Product(@Id Long id, String name, BigDecimal price) {}

// Repository – ZERO implementation needed!
public interface ProductRepository
    extends CrudRepository<Product, Long> {
    List<Product> findByNameContaining(String keyword);

    @Query("SELECT * FROM product WHERE price < :max ORDER BY price")
    List<Product> findCheaperThan(@Param("max") BigDecimal max);
}

// Service
@Service @Transactional
public class ProductService {
    public Product create(String name, BigDecimal price) {
        return repo.save(new Product(null, name, price));
    }
}
```

4 · Hibernate & Jakarta Persistence (JPA) — Introduction

Hibernate is the most popular JPA provider. Jakarta Persistence (formerly Java EE JPA) is the specification; Hibernate is the implementation. Together they form a full Object-Relational Mapping (ORM) layer.

Jakarta Persistence (Specification)

- `@Entity`, `@Table`, `@Id`, `@Column` annotations
- JPQL — object-oriented query language
- EntityManager lifecycle management
- Defines the contract (interface)
- Implemented by: Hibernate, EclipseLink, OpenJPA

Hibernate (Implementation)

- Implements all JPA contracts
- Adds: 2nd-level cache, batch inserts
- Lazy loading of associations
- HQL + Criteria API + native SQL
- Session factory & persistence context

Key ORM concepts: Entity identity • Persistence context • First-level cache • Dirty checking • Proxy objects for lazy loading • Cascade types

5 · Spring Data JPA — Introduction

Spring Data JPA sits on top of Jakarta Persistence / Hibernate. It auto-generates repositories, reduces boilerplate even further, and adds Pageable, Specification, and auditing out-of-the-box.

Spring Data JDBC vs. Spring Data JPA

Feature	Spring Data JDBC	Spring Data JPA
Underlying engine	JdbcTemplate	Hibernate (JPA provider)
Session / Persistence ctx	None	EntityManager — tracks entities
Lazy loading	No	Yes — @ManyToOne, @OneToMany
1st-level cache	No	Yes (within transaction)
2nd-level cache	No	Yes (e.g. Ehcache, Redis)
DDL generation	Via schema.sql	@Entity → auto schema (hbm2ddl)
Pagination	Manual	Pageable / Page<T> built-in
Auditing	Manual	@CreateDate / @LastModifiedDate
Complexity	Low	Medium-High
Best for	Simple CRUD, microservices	Complex domains, rich relationships

5 · Spring Data JPA — Maven, Config & Code

pom.xml (Maven)

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
</dependency>
```

application.properties

```
spring.datasource.url=jdbc:postgresql://localhost:5432/mydb
spring.datasource.username=postgres
spring.datasource.password=secret
# DDL: validate | update | create-drop | none
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.properties.hibernate.jdbc.batch_size=25
```

```
@Entity @Table(name = "product")
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    @Column(precision=10, scale=2)
    private BigDecimal price;
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "category_id")
    private Category category; // lazy-loaded
    // getters/setters or Lombok @Data
}

public interface ProductRepository
    extends JpaRepository<Product, Long> {
    Page<Product> findByCategory(Category c, Pageable p);

    @Query("SELECT p FROM Product p WHERE p.price < :max")
    List<Product> findCheaper(@Param("max") BigDecimal max);
}
```

6 · Full Technology Comparison

Criterion	Vanilla JDBC	JdbcTemplate	Spring Data JDBC	Spring Data JPA
SQL control	Full	Full	Partial (@Query)	Partial (JPQL/@Query)
Boilerplate	Very high	Low	Minimal	Minimal
ORM mapping	None	Manual RowMapper	Auto (@Table)	Full ORM (@Entity)
Lazy loading	N/A	N/A	No	Yes
Caching	None	None	None	L1 + L2 cache
Session mgmt	Manual	None	None	EntityManager
DDL generation	None	None	schema.sql	Hibernate auto
Relationships	Manual JOINS	Manual JOINS	AggregateRef	Full JPA relations
Pagination	Manual	Manual	Manual/Pageable	Pageable/Page<T>
Transactions	Manual	@Transactional	@Transactional	@Transactional
Test difficulty	Easy mock	Easy	Easy	Requires context
Learning curve	Low	Low	Low	Medium-High
Performance	Best	Very good	Good	Good (tunable)
Best use case	Legacy, tuned SQL	Microservices, CQRS	Simple CRUD apps	Complex enterprise apps

Technology Stack — Architecture Overview

Spring Data JPA

JpaRepository / @Entity

Highest abstraction
Full ORM — entities, relationships,
caching, pagination

Spring Data JDBC

CrudRepository / @Table

Lightweight ORM
DDD-friendly, no lazy loading,
predictable queries

Spring JdbcTemplate

JdbcTemplate / NamedParameterJdbcTemplate

SQL-first helper
Removes JDBC boilerplate,
full SQL control

Jakarta Persistence (JPA API)

EntityManager / JPQL

Standard spec
Portable across JPA providers
(Hibernate, EclipseLink...)

Hibernate ORM

JPA Provider — Dialect, Cache, Proxy

De-facto ORM provider
HQL, batch processing,
2nd-level cache

JDBC API

Connection / PreparedStatement / ResultSet

JDBC Driver + Database

PART 2

Performance Analysis

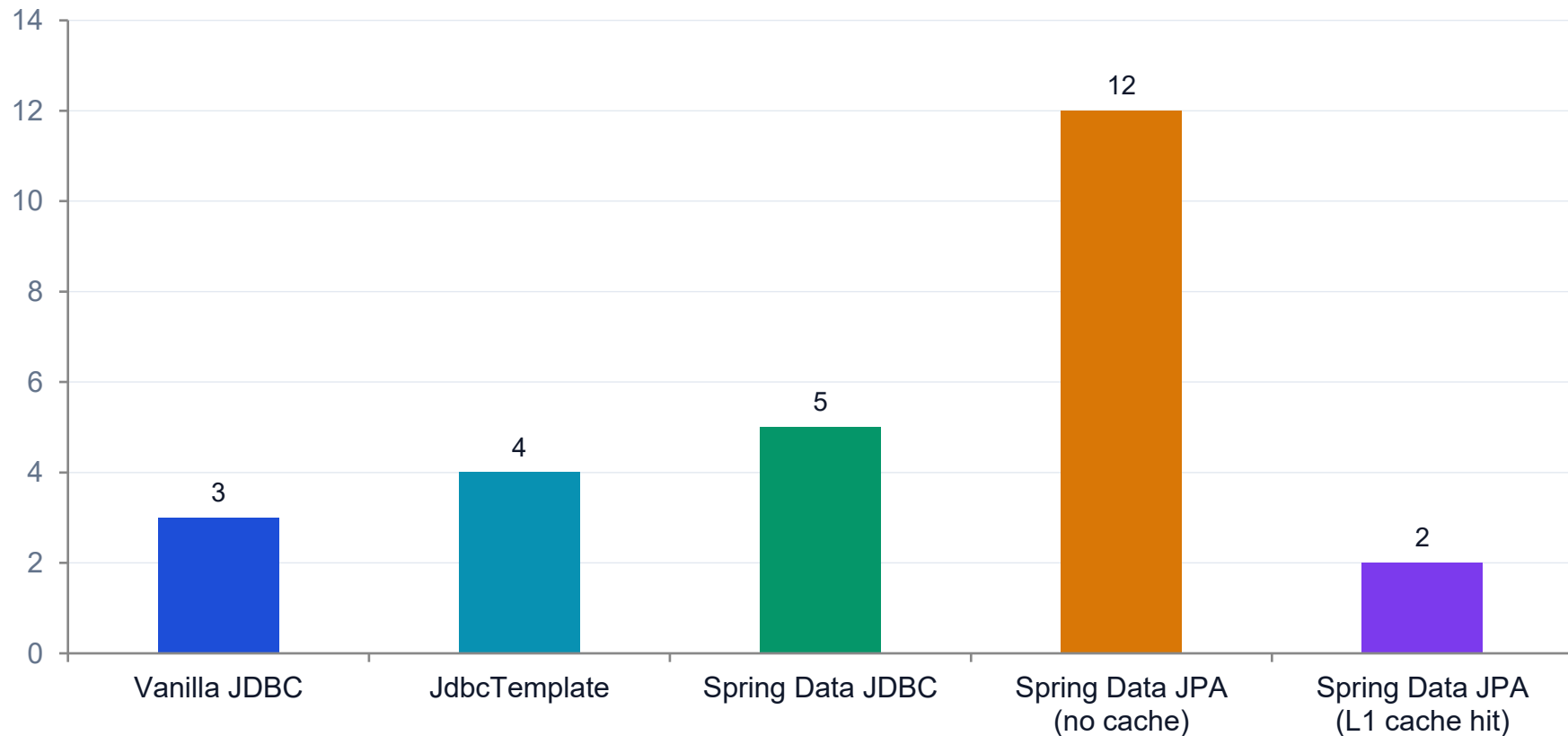
7 · Benchmark Methodology

Representative benchmark conditions for Java database access technologies (PostgreSQL, Spring Boot 3.x, JVM warm-up 3 runs, average of 10 runs, HikariCP pool size 10).

Parameter	Small Dataset	Large Dataset
Row count	100 – 1,000 rows	100,000 – 1,000,000 rows
Operations measured	SELECT, INSERT, UPDATE, DELETE	SELECT, INSERT, UPDATE, DELETE
Metric	Milliseconds (ms)	Milliseconds (ms) / throughput
Object complexity	Flat entity (5 fields)	Entity + 2 relations, 10 fields
Transaction scope	Per-operation	Batch (1000 ops/tx for large)
JVM	OpenJDK 21 (LTS)	OpenJDK 21 (LTS)
Database	PostgreSQL 16, local	PostgreSQL 16, local
Connection pool	HikariCP default	HikariCP max-pool=20

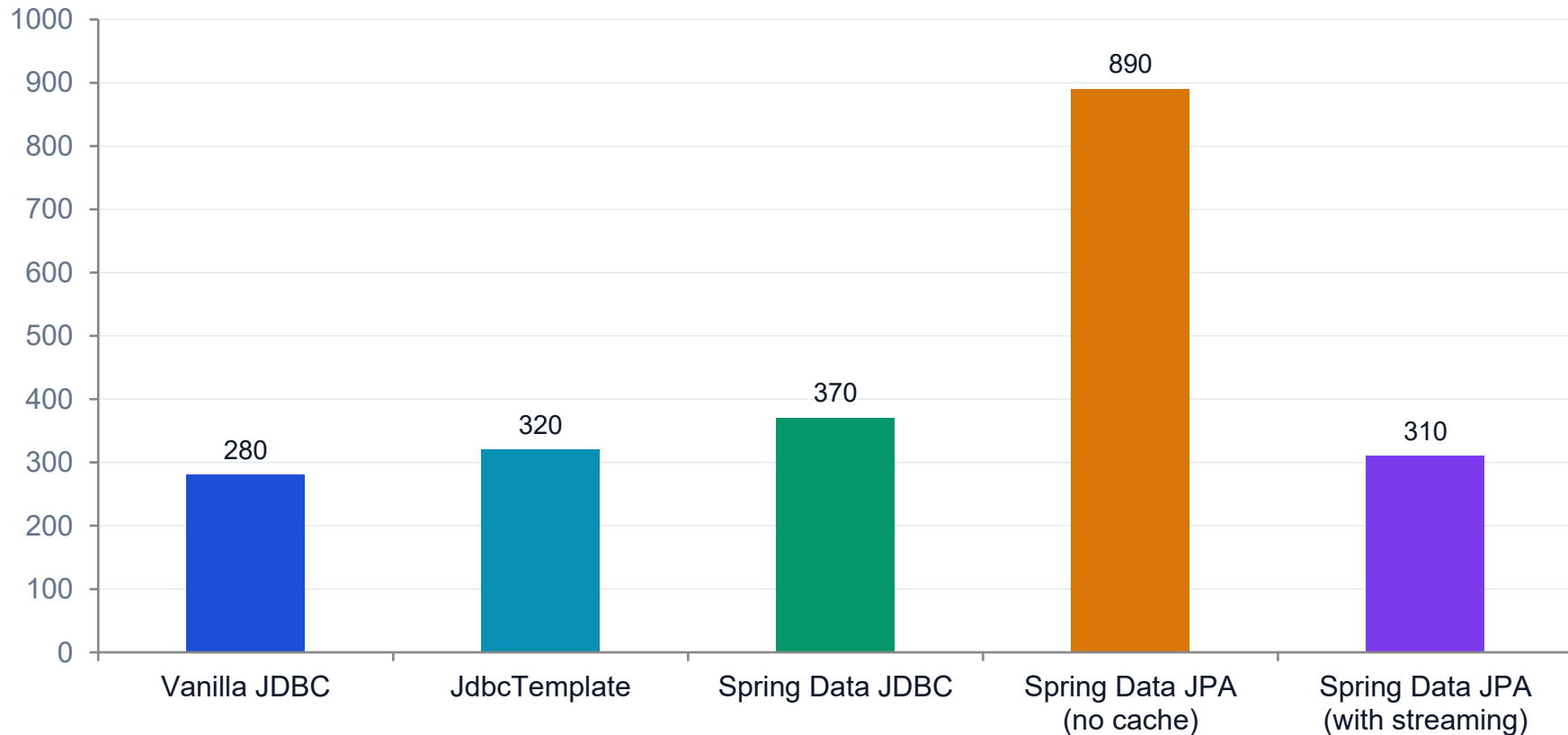
⚠ Note: These are representative figures from published benchmarks and typical production profiles. Always measure in YOUR specific environment.

8 · READ Performance — Small Dataset (1,000 rows, avg ms)



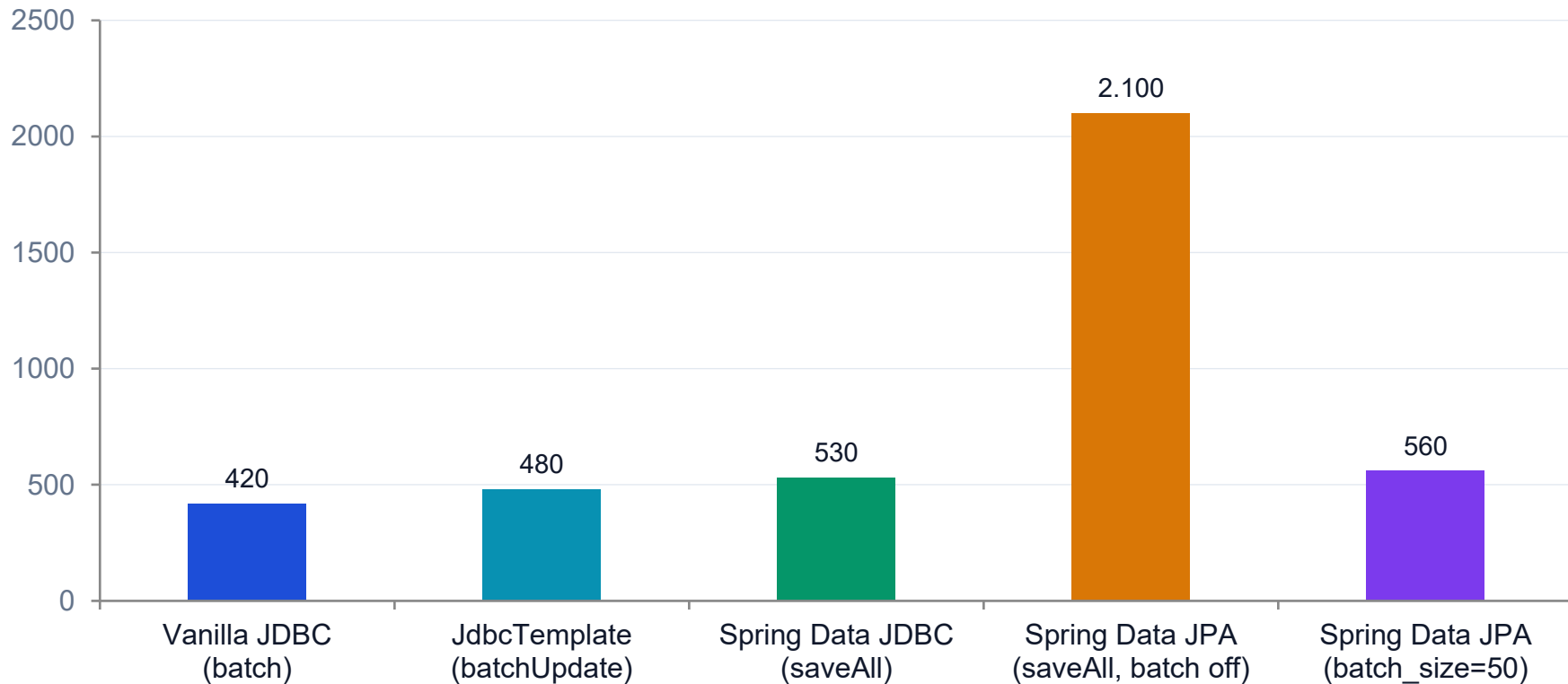
Lower is better · Single-row lookup by primary key · Connection pool warmed up

8 · READ Performance — Large Dataset (100,000 rows fetch-all, ms)



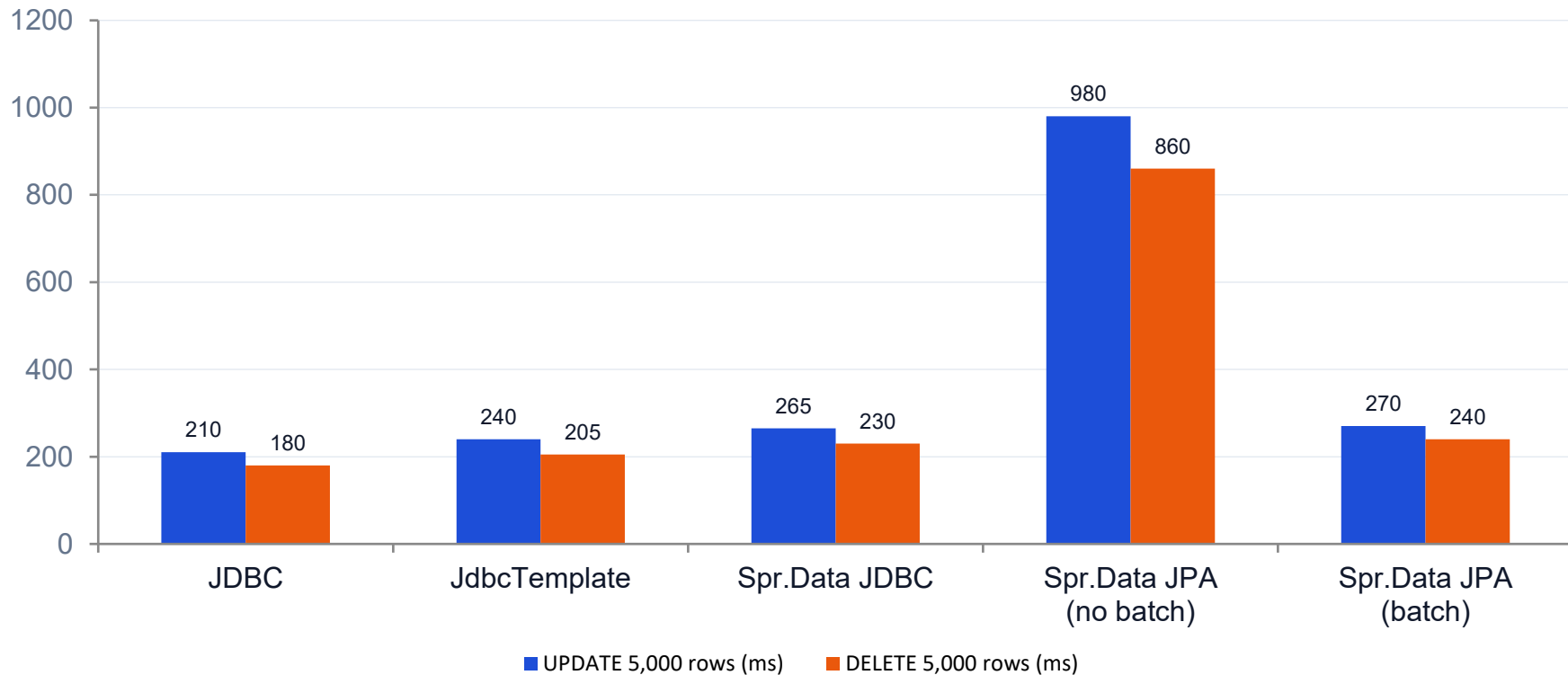
JPA object hydration + persistence context overhead is significant at large scale · Use streaming / projections for large reads

8 · WRITE (INSERT) Performance — 10,000 records (ms total)



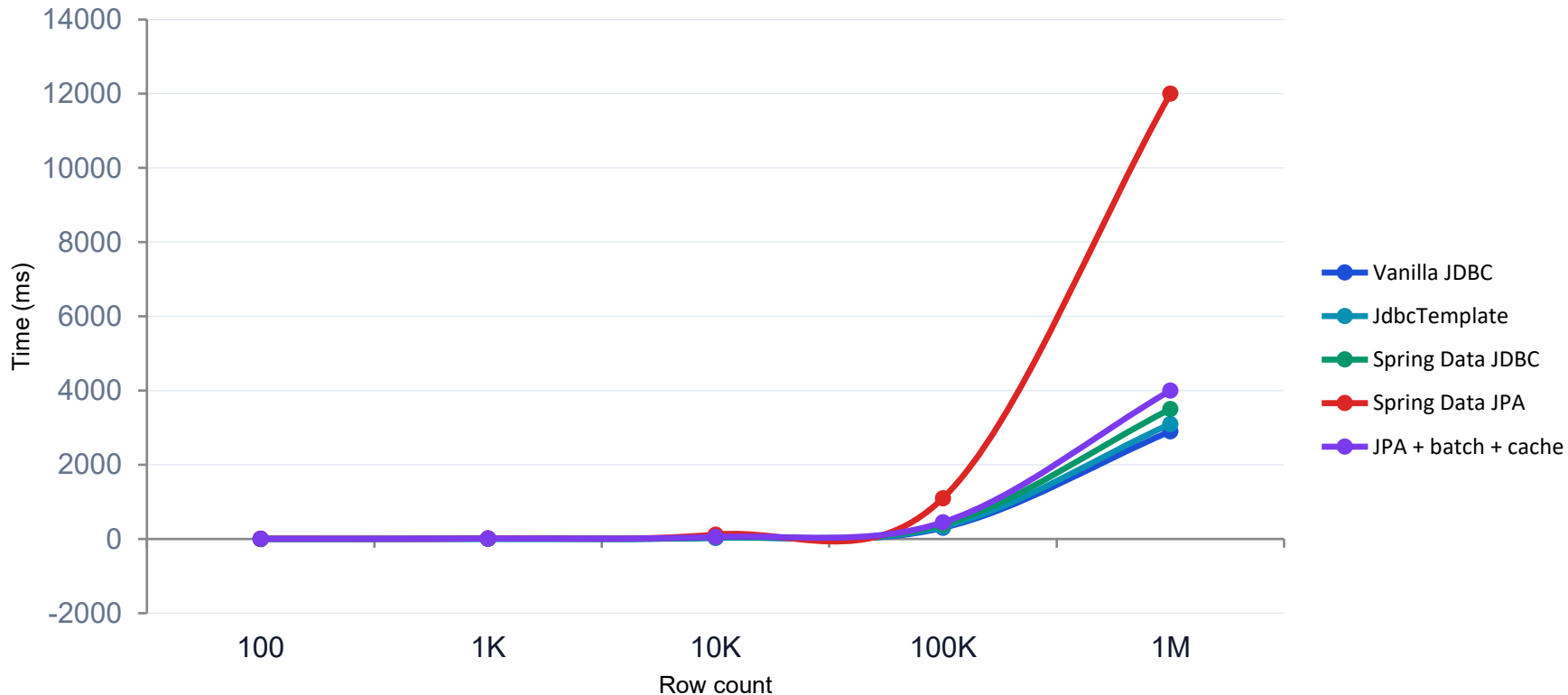
⚠ JPA without batch_size configured is dramatically slower — always set `hibernate.jdbc.batch_size` for bulk writes!

8 · UPDATE & DELETE Performance — 5,000 records (ms)



DELETE in JPA loads each entity into persistence context before deletion (dirty check) — use @Query + JPQL bulk delete for mass deletes

9 · Performance Scaling: Small vs Large Dataset



Row count (log scale) → JPA overhead grows super-linearly without batch/cache tuning

9 · Performance Summary Table

Operation / Scale	Vanilla JDBC	JdbcTemplate	Spring Data JDBC	Spring Data JPA (default)	Spring Data JPA (tuned)
Single row READ (small)	★★★★★	★★★★★	★★★★☆	★★★☆☆	★★★★★
Bulk READ (large)	★★★★★	★★★★☆	★★★★☆	★★★☆☆	★★★★☆
Bulk INSERT (large)	★★★★★	★★★★★	★★★★☆	★★★☆☆	★★★★☆
Bulk UPDATE (large)	★★★★★	★★★★★	★★★★☆	★★★☆☆	★★★★☆
Bulk DELETE (large)	★★★★★	★★★★★	★★★★☆	★★★☆☆	★★★★☆
Developer productivity	★★★☆☆	★★★★☆	★★★★★	★★★☆☆	★★★★★

Green = excellent · Yellow = acceptable · Red = needs attention / configuration

JPA Performance — Key Tuning Tips

Enable JDBC Batching

Set `hibernate.jdbc.batch_size=25-50` in `application.properties`. Without it every INSERT is a separate round-trip — 10x slower for bulk ops.

Use Projections for Reads

Instead of loading full `@Entity` graphs, use interface projections or `@NamedNativeQuery` with DTOs to avoid N+1 and reduce object hydration.

Avoid N+1 Selects

Use JOIN FETCH in JPQL or `@EntityGraph` to eagerly load associations in a single query. N+1 is the #1 JPA performance killer.

2nd-Level Cache

Enable Hibernate 2nd-level cache (Ehcache/Redis) for read-heavy, rarely-updated entities. Huge impact on repeated reads.

Bulk Operations

Use `@Modifying @Query("UPDATE/DELETE ...")` for mass changes instead of loading entities. Bypasses persistence context overhead.

FetchType Strategy

Default `@OneToMany` to LAZY. Only fetch eagerly when the association is ALWAYS needed. Eager on collections = performance disaster.

10 · Decision Guide — When to Use What

Vanilla JDBC

Performance-critical systems, legacy codebases, batch ETL pipelines, DBA-written queries that must not be touched, sub-millisecond latency requirements.

JdbcTemplate

Microservices with simple CRUD, CQRS read-side, systems where SQL control matters but boilerplate elimination is desired. Great starting point for new teams.

Spring Data JDBC

DDD-based architectures, clean aggregate design, microservices with moderate domain complexity. No lazy loading = predictable performance. Best default choice.

Spring Data JPA (Hibernate)

Complex enterprise domains with rich relationships (bidirectional, many-to-many, inheritance), auditing, advanced caching. Requires team expertise. Tune batch + cache.

Rule of thumb: Start with Spring Data JDBC. Add JPA when domain complexity demands it. Drop to JdbcTemplate when you need SQL control. Use JDBC only for extreme performance.

Conclusion — Key Takeaways for Students

JDBC

Foundation of all Java DB access. Understand it deeply — everything builds on it.

Jakarta Persistence

The standard specification. Learning it makes you portable across JPA providers.

JdbcTemplate

Your first step up. Eliminates boilerplate, preserves full SQL control.

Hibernate

The most powerful JPA provider. Understand lazy loading, cache, and batch config.

Spring Data JDBC

Recommended default for new microservices. Simple, predictable, DDD-ready.

Spring Data JPA

Most productive for complex domains. Configure properly or it will be slower.

Questions?

Java Web Programming
Algebra Bernays University

JDBC · JdbcTemplate · Spring Data JDBC · Hibernate · Spring Data JPA